Dzanga-Sangha Simulation
Program Flow & Logic Summary

Based on dzanga_sim(1).py | Python port of large_scale_sim_FINAL_ver_5_REAL.m

Purpose and architecture

The program models elephant movement, poacher activity, acoustic-microphone detection, and ranger intervention inside a mapped park boundary. DzangaSimulation owns the model and all simulation state; DzangaApp provides a Tkinter dashboard. An alternative command-line headless mode runs a fixed number of time steps and prints results.

Model setup

Initialization builds the park occupancy grid, five village avoidance zones, the Bai attractor/protected zone, 20 elephants, 10 poachers, position-sampling pools, and microphones. Defaults are 80 microphones using Strategy 1. Optional random seed enables reproducible stochastic runs.

Stage	What the code does
1 · Uniform spread	Farthest-point placement across candidate park locations.
2 · Targeted fortress	Put most microphones around village rings, then spatially fill remaining sites.
3 · Perimeter defense	Space microphones along park-boundary segments.
4 · Combination	Combine village-ring and perimeter positions, then fill any remainder.

Sequence of one simulation step

Stage	What the code does
1 · Advance time	Multiply elapsed real time by the chosen acceleration, then compute per-agent travel distances.
2 · Move poachers	Travel toward roaming targets; avoid the Bai region; turn back or retarget at park boundary.
3 · Move elephants	Transition through ROAMING, SEEKING, and WAITING; move toward Bai when appropriate; avoid village zones and boundaries.
4 · Scan microphones	At most one scan per microphone per simulated second; evaluate nearby elephants and poachers,

	missed detections, and elephant memory.
5 · Dispatch/move rangers	When microphone evidence identifies a threat, target poachers and send a ranger from the nearest village; catch them within 500 m.
6 · Evaluate encounters	Check proximity outside Bai, apply one stochastic poaching roll per continuous encounter, update alert flags and counters.

Important execution detail: within step(), the source calls _move_poachers(), _move_elephants(), _scan_mics(), _move_rangers(), then _poaching_check(), in exactly that order.

Figure 1 · Overall program flow

Orthogonal connectors show command-line mode selection, model construction, the Tkinter timer loop, and headless execution. The GUI updates the model only when it is not paused.

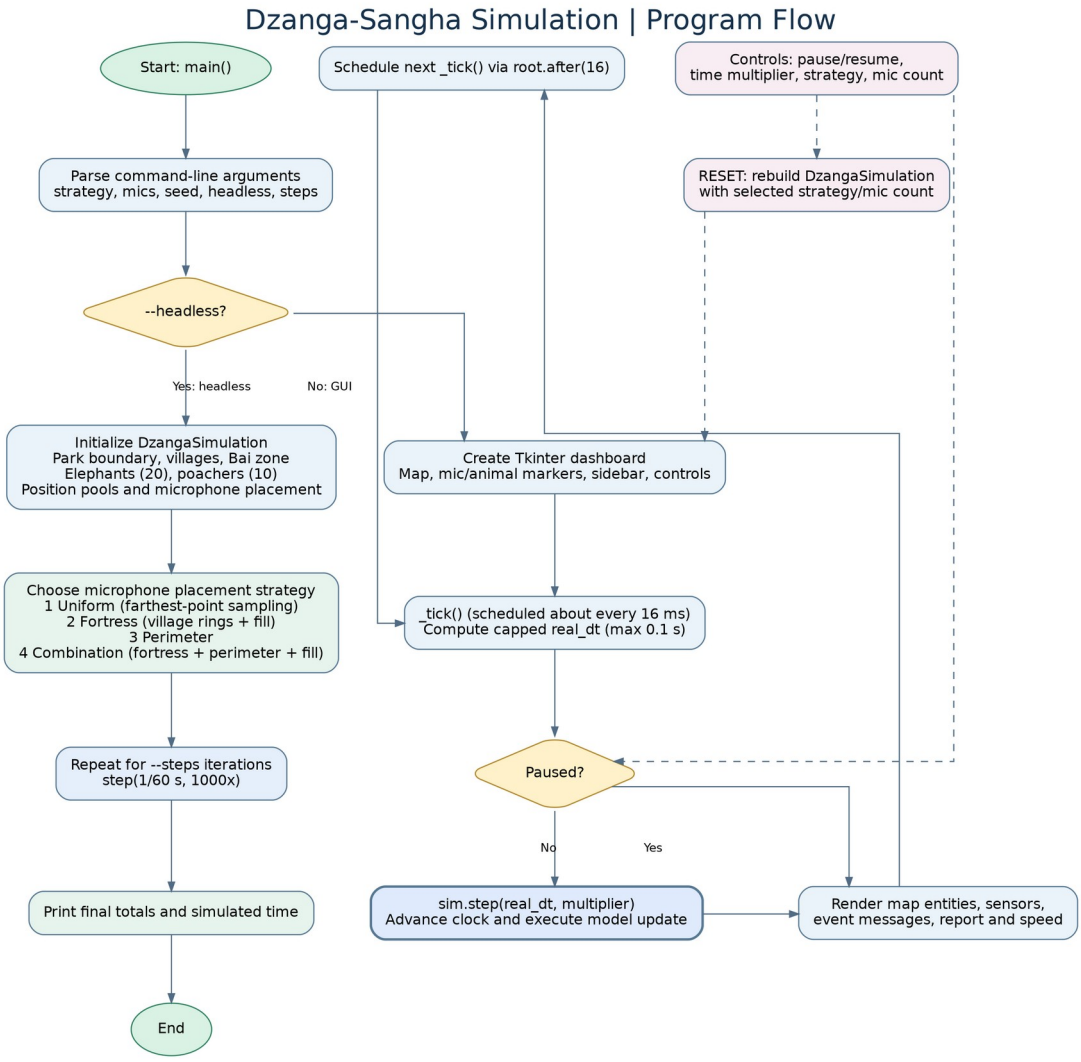
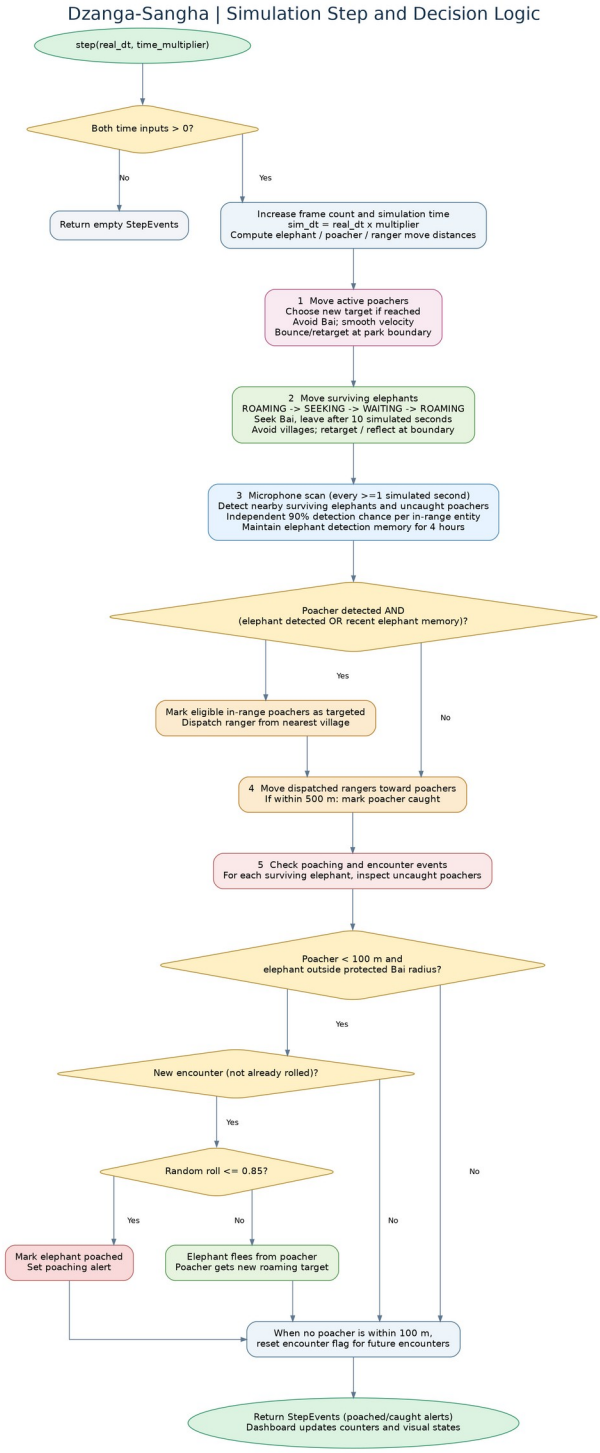


Figure 2 · Per-step model decisions

This diagram follows DzangaSimulation.step(), with branches for invalid time input, threat detection, ranger dispatch, and elephant–poacher encounters.



Key parameters and interpretation

Stage	What the code does
Detection	90% independent random chance when an eligible entity is within a microphone's applicable detection radius.
Sensor ranges	Elephant: 3,950 m; poacher: 200 m.
Sensor timing	Each microphone scans after ≥1 simulated second since its last scan; elephant memory lasts 14,400 s (4 h).
Poaching event	Poacher within 100 m of a living elephant outside the Bai protected radius; 85% chance on a new eligible encounter.
Ranger interception	Ranger is sent from nearest village when an in-range poacher is detected together with an elephant detection or recent memory; capture threshold is 500 m.
Model speeds	Elephant 1.11 m/s; poacher 0.55 m/s; ranger 5.55 m/s.
GUI / headless	GUI _tick() scheduled at approximately 16 ms; headless defaults to 500 iterations at 1/60 s real time and 1000× multiplier.

Behavioral nuances

A microphone alone does not automatically dispatch a ranger: a detected poacher must coincide with a current elephant detection or valid elephant memory at that microphone. Ranger targets remain targeted once set. Poaching trials are not repeated on every frame during a continuous close encounter; the encounter flag resets when no uncaught poacher is within 100 m.

The Bai is protected from the poaching event check, but it is not a universal safety guarantee against all interactions or movement. The simulation models stochastic outcomes, not measured real-world intervention effectiveness.

Source and scope

Source: uploaded dzanga_sim(1).py, especially DzangaSimulation.__init__, _place_mics, step, _scan_mics, _move_rangers, _poaching_check, DzangaApp._tick, run_headless, and main. Flowcharts are simplified conceptual depictions of these control paths; this report does not claim to validate model realism or predictions.